



Neural Ordinary Differential Equations

and Their Application to Epidemic Spreading on Networks

Bachelor Seminar

What If Neural Network Had No Layers?

Table of Contents

1

ODE Foundations

What an ODE is, IVPs and solvers, vector fields, the bridge to deep networks

2

Neural ODE Architecture

Core formulation, what the network learns, adaptive solvers, training

3

Case Study: Epidemic Spreading

Problem setup, SIR model, intractability, existing approximations and their trade-offs

4

The GN-ODE Model

Approximate ODE core, neural architecture, datasets, and baselines

5

Results And Conclusions

Prediction accuracy, scalability, generalization to unseen settings, and final takeaway

ODE Foundations

What an ODE is · Initial value problems and solvers · Vector fields · The bridge from Euler steps to deep networks

Ordinary Differential Equations

An Ordinary Differential Equation (ODE) describes how a quantity **changes over time**, not its absolute value at each point. → ODE tells us the direction of motion.

$$\frac{dy(t)}{dt} = f(y(t), t)$$

- $y(t)$: state of the system at time t
- $\frac{dy(t)}{dt}$: rate of change
- $f(y(t), t)$: rule that tells the system how to evolve



Key distinction from algebraic equations: the unknown is a function, not a number

An ODE is called “ordinary” because derivatives are taken with respect to one independent variable – time.

Examples: population growth, radioactive decay, pendulum motion



Simple scalar ODE: Exponential Decay

Idea: a quantity decreases at the rate proportional to its current value, i.e.

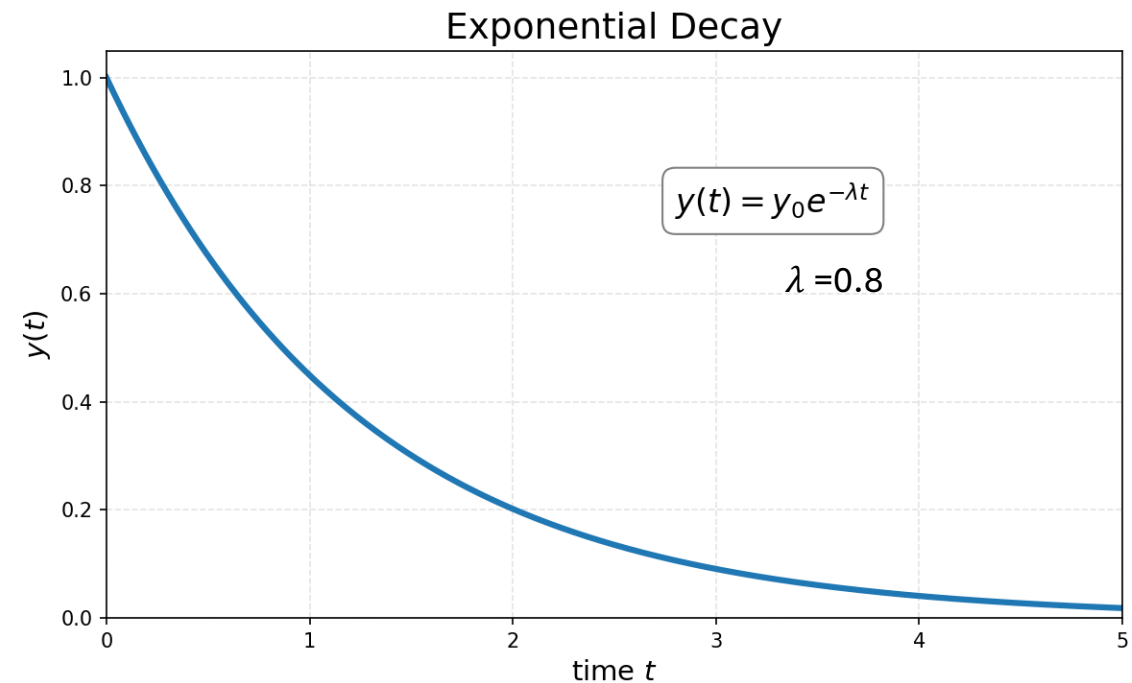
$$\frac{dy(t)}{dt} = -\lambda y(t)$$

- $y(t)$: current amount
- $\lambda > 0$: decay rate
- larger $y(t) \rightarrow$ faster decrease
- smaller $y(t) \rightarrow$ slower decrease

The **solution** is:

$$y(t) = y_0 e^{-\lambda t}$$

*can be obtained by the method “separation of variables”.



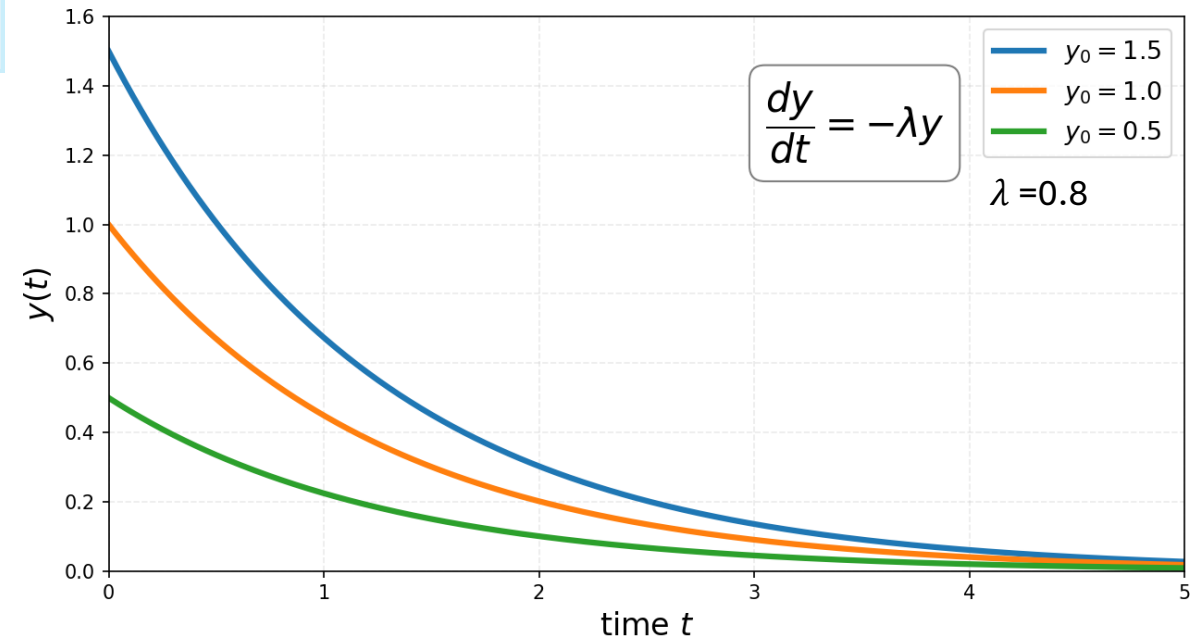
From Rule of Change to Prediction



An ODE gives the rule of motion, but the initial value tells us where the motion starts.

Initial Value Problem (IVP) $\frac{dy}{dt} = f(y, t)$
 $y_0 = y(t_0)$

- The **rule**: $f(y, t)$ tells the system how to change.
- The **starting point**: y_0 fixes the initial state.
- The **trajectory**: $y(t)$ is what we get after following the rule over time.



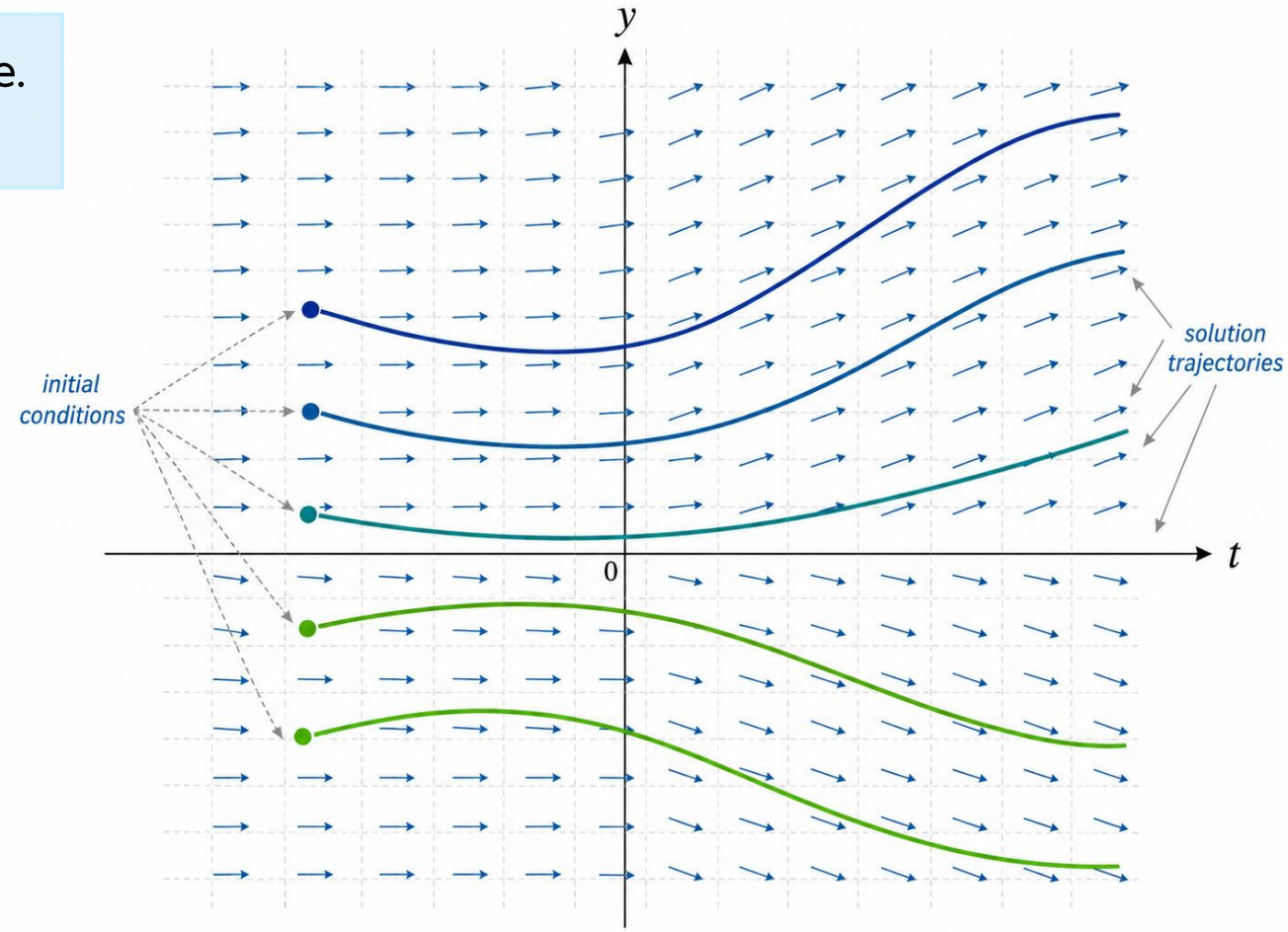
Same rule of change, different initial values
→ different trajectories.

Vector Fields: The ODE's Direction Map



An ODE does not directly give the whole curve.
It gives a local direction at every point.

- Each point (t,y) has an arrow.
- Arrow direction = where the state moves next.
- Arrow length = speed of change.
- A trajectory is obtained by starting at y_0 and following the arrows.



Approximating Solution Using Euler's Method



Closed-form solutions are rare. In practice, one often approximates solution (trajectory) step by step.

Euler's method (Explicit/Forward)

Start at $y(t)$, evaluate the derivative, take a step forward.

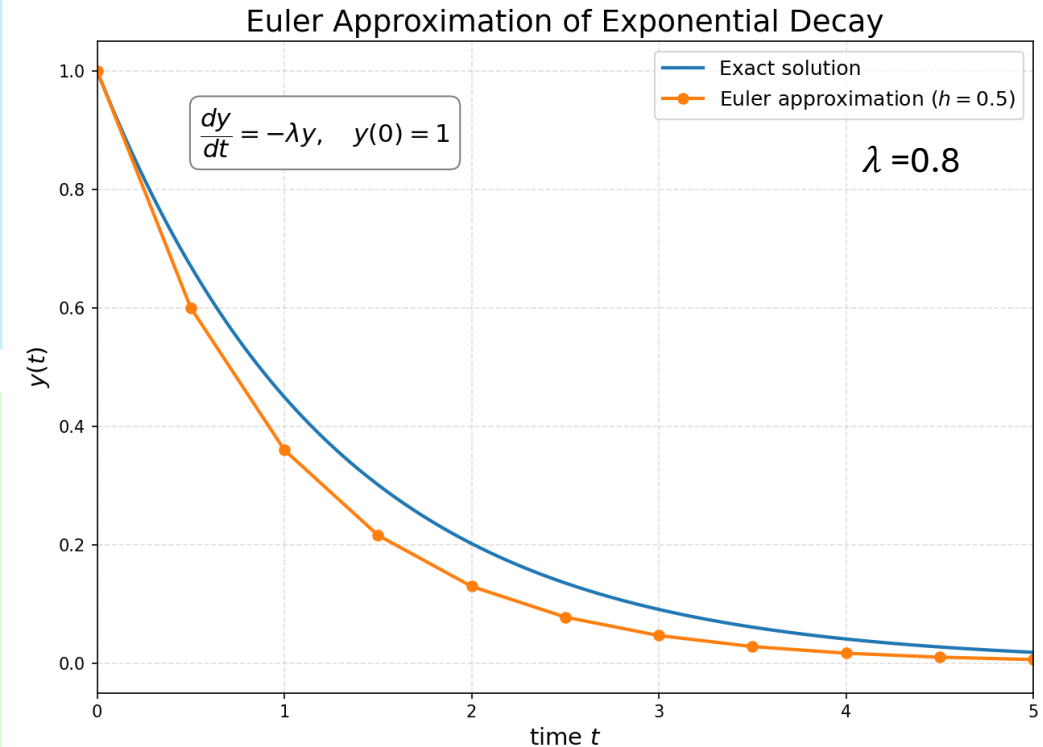
$$y_{n+1} = y_n + hf(y_n, t_n),$$

where h is the step size.

For exponential decay $\frac{dy}{dt} = f(y, t)$, Euler's update becomes:

$$y_{n+1} = y_n - h\lambda y_n$$

So each step moves the state downward, approximating the smooth decay trajectory.



Approximating Solution Using Euler's Method



Closed-form solutions are rare. In practice, one often approximates solution (trajectory) step by step.

Euler's method (Explicit/Forward)

Start at $y(t)$, evaluate the derivative, take a step forward.

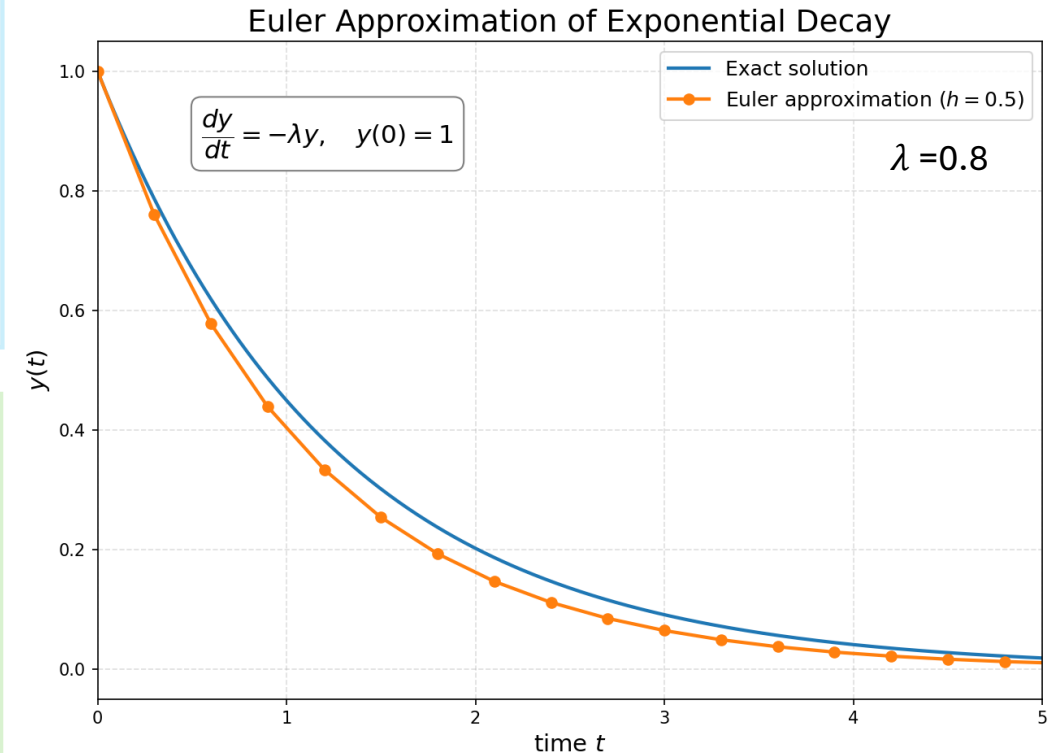
$$y_{n+1} = y_n + hf(y_n, t_n),$$

where h is the step size.

For exponential decay $\frac{dy}{dt} = f(y, t)$, Euler's update becomes:

$$y_{n+1} = y_n - h\lambda y_n$$

So each step moves the state downward, approximating the smooth decay trajectory.



From Euler Steps to Neural Networks

Euler's method

$$y_{n+1} = y_n + \Delta t f(y_n, t_n)$$

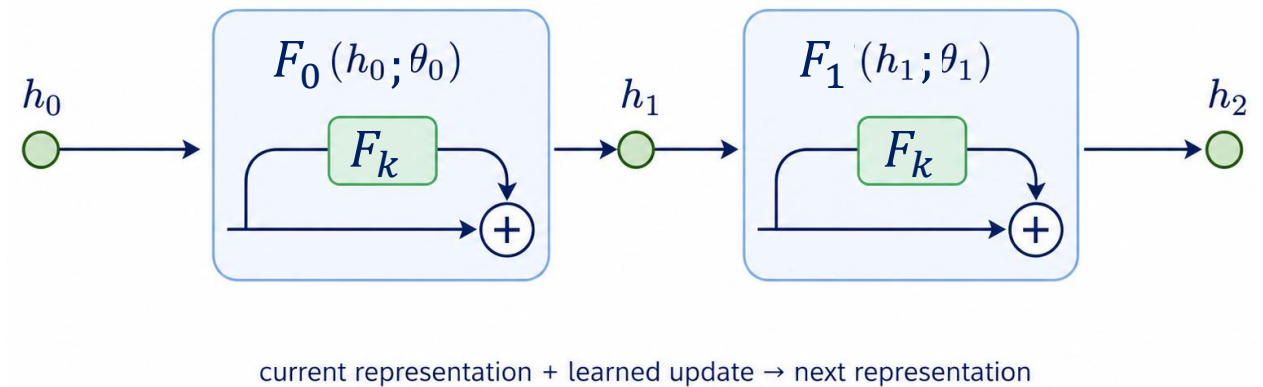
Residual Neural Network

$$h_{k+1} = h_k + F_k(h_k; \theta_k)$$

- h_k is the hidden representation at layer k
- F_k is the function learned by block k
- $F_k(h_k; \theta_k)$ is the learned update that gets added to the current representation

* Δt is usually absorbed into F_k or treated as 1.

$$h_{k+1} = h_k + F_k(h_k; \theta_k)$$



This is the **key observation** used in Neural ODEs: residual networks can be viewed as Euler-style discretization of continuous dynamics.

Neural ODE Architecture

Core formulation · What the network learns · Adaptive solvers · Training

The Neural ODE: Core Formulation

A standard deep network applies a sequence of layers:

$$h_0 \rightarrow h_1 \rightarrow \dots \rightarrow h_L$$

A Neural ODE replaces this discrete layer stack with a continuous hidden-state evolution:

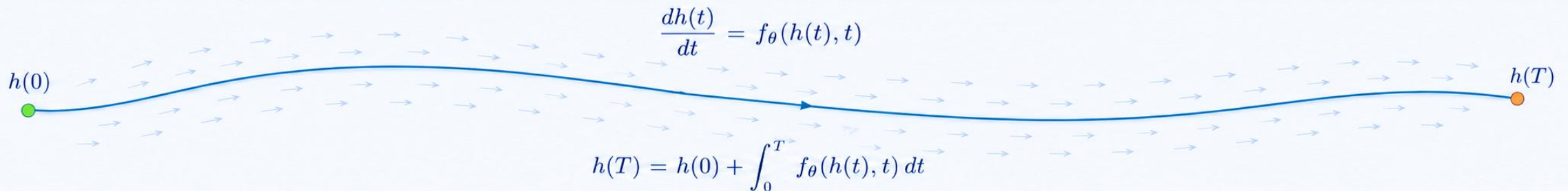
$$\frac{dh(t)}{dt} = f_{\theta}(h(t), t, \theta)$$

Here, f_{θ} is a neural network that learns the dynamics of the hidden state.

The final representation is obtained by solving an IVP:

$$h(T) = h(0) + \int_0^T f_{\theta}(h(t), t) dt$$

Or equivalently: $h(T) = \text{ODESolve}(h(0), f_{\theta}, 0, T)$

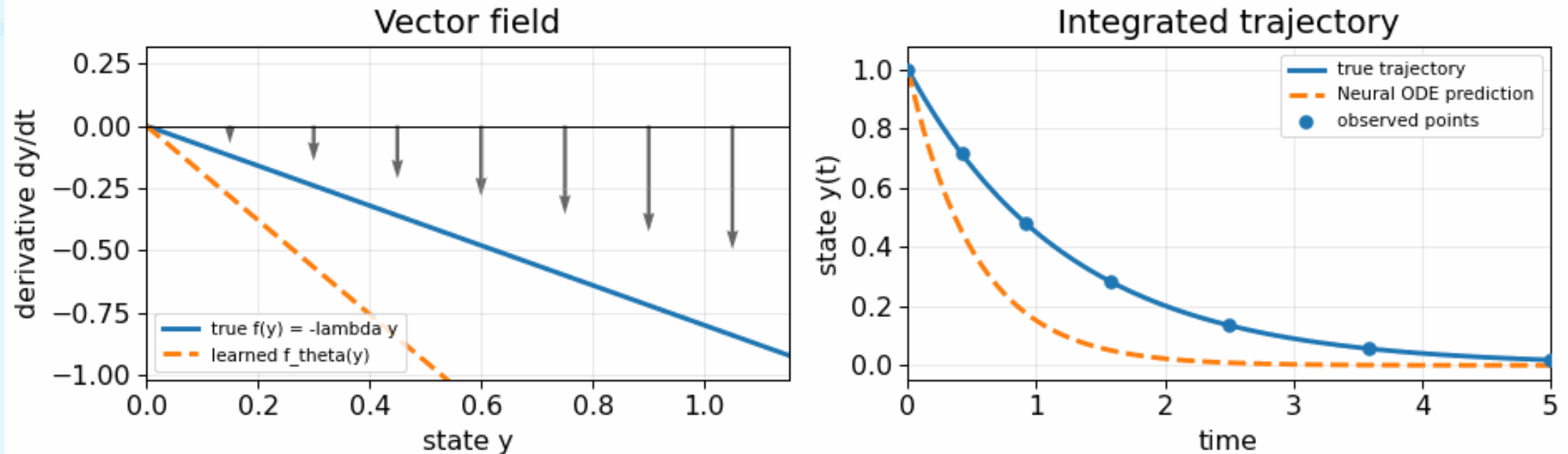


What the Network **Actually** Learns

The network learns a vector field $f_{\theta}(h(t), t)$ which tells the hidden state how to change at each point.

- It learns the local direction of motion. The ODE solver turns these local directions into a full trajectory.
- All input share the same field f , only different initial conditions – contrast with ResNet.

Learning a vector field for exponential decay



epoch: 0 learned theta: -1.889 target theta: -0.800 loss: 3.28e+00

The Solver as an Adaptive Compute Engine

The neural network gives the derivative:

$$f_{\theta}(y, t)$$

The solver decides how to approximate the trajectory.

- It evaluates f_{θ} at selected time points.
- Step sizes can be adjusted automatically.
- More evaluations are used when the trajectory is harder to follow.
- A tolerance controls the accuracy/speed tradeoff.



The model learns the dynamics; the solver handles numerical integration.

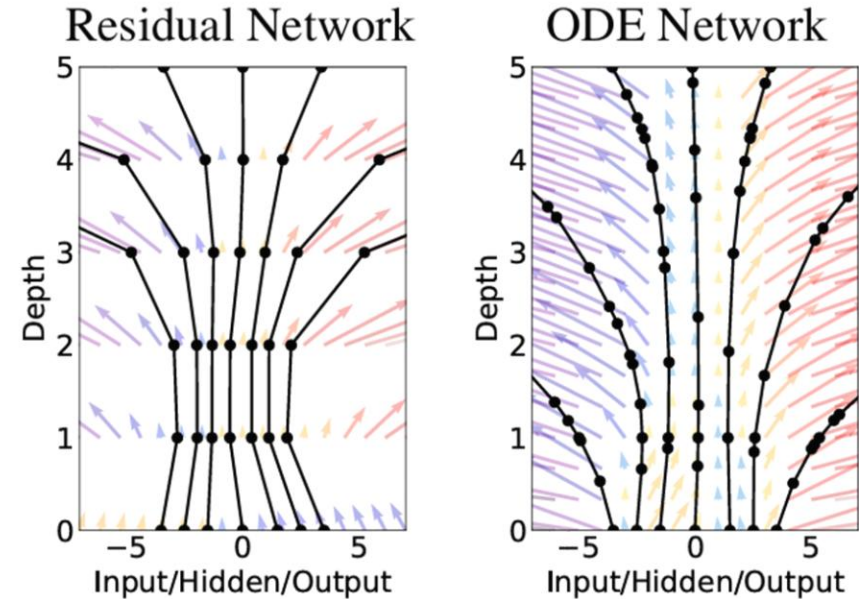


Figure 1: *Left:* A Residual network defines a discrete sequence of finite transformations. *Right:* A ODE network defines a vector field, which continuously transforms the state. *Both:* Circles represent evaluation locations.

Training a Neural ODE: Backpropagation Through the Trajectory

1

Forward pass

$$h(T) = \text{ODESolve}(h(0), f_{\theta}, 0, T)$$

The solver follows the learned dynamics and produces the final representation $h(T)$. Then use $h(T)$ to make a prediction $\hat{y}$.

2

Loss

$$L = \text{loss}(\hat{y}, y)$$

The loss measures how far the prediction is from the target.

3

Backward pass

$$a(t) = \frac{\partial h(t)}{\partial L}$$

The error is propagated backward through the trajectory.

How much did each hidden state $h(t)$ influence the final loss?

This gives the gradient information needed to improve the dynamics f_{θ} .

4

Parameter update

$$\theta \leftarrow \theta - \eta \frac{\partial \theta}{\partial L}$$

The trajectory itself is not changed during backpropagation.

Instead, f_{θ} is adjusted so that the next forward pass produces a better trajectory.

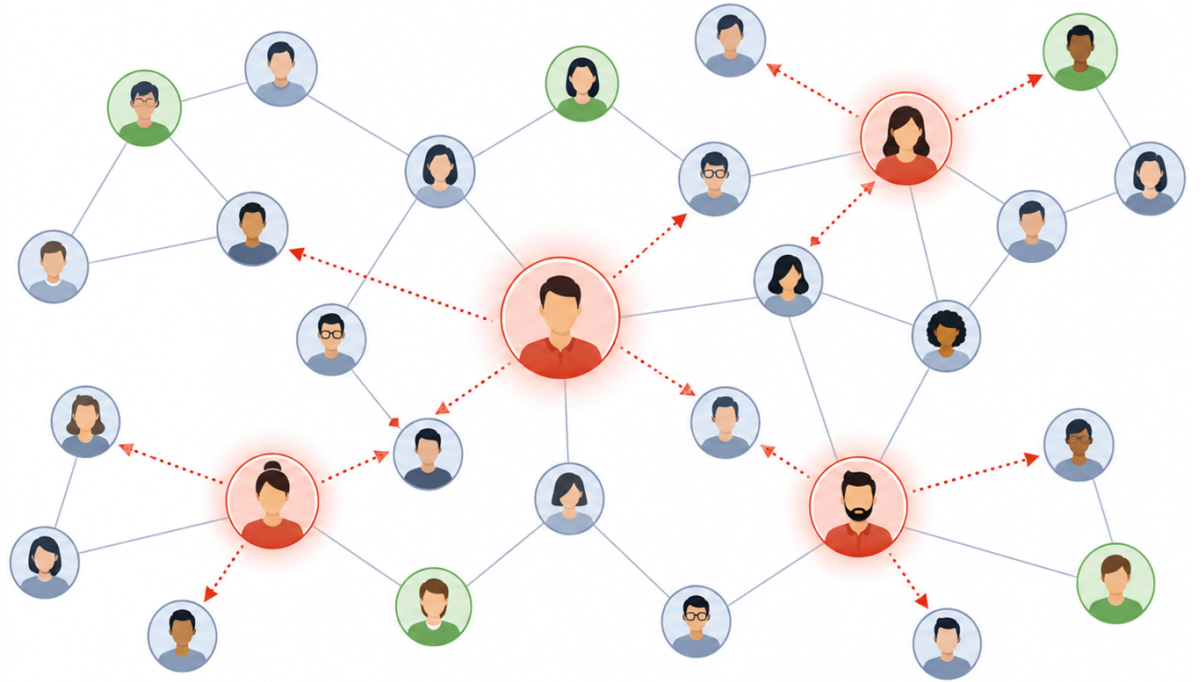
Note: The original Neural ODE paper trains the model using the [adjoint sensitivity method](#). Instead of backpropagating through every internal step of the ODE solver, it treats the solver as a black box and solves an augmented ODE backward in time to compute gradients.

Case Study: Epidemic Spreading

Problem setup, why model on networks · The SIR model · Why exact computation is intractable · Existing approximations and their tradeoffs

Epidemic Spreading as a Real-World Complexity Problem

- **Real-world system:** many interacting states evolve over time.
- **Partly known:** we know the basic infection/recovery mechanism.
- **Network-structured:** one node's evolution depends on its neighbors.
- **Computationally hard:** exact prediction becomes infeasible on large graphs.

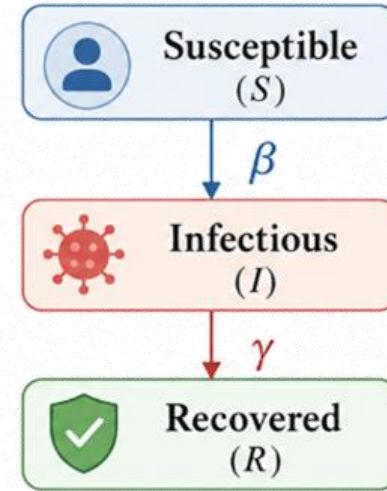


Why Classical Solvers Fail?

- Transmission changes with behavior: masks, lockdowns, distancing.
- Contact intensity varies across places and communities.
- Reporting is delayed, noisy, and irregular.
- Simple equations only approximate the real process.

The SIR Model

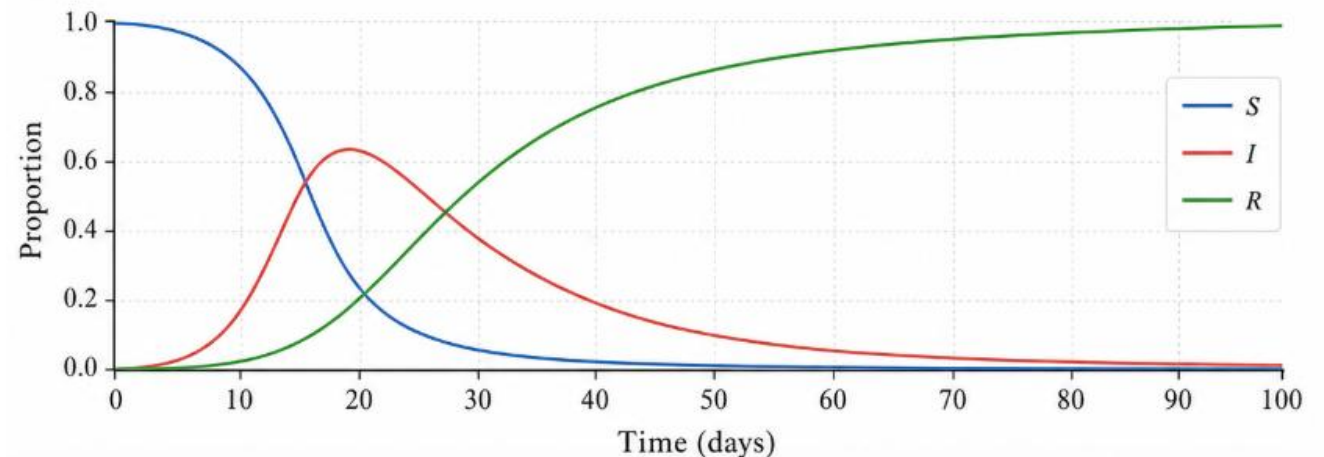
- Each node is in one of **three states**: Susceptible (S), Infected (I), or Recovered (R).
- β controls infection: a susceptible node can become infected through contact with an infected neighbor.
- γ controls recovery: an infected node moves to the recovered state.
- Assumptions:** after recovery, the node is assumed to be immune; β, γ are uniform.
- Goal:** predict the probabilities $P(S)$, $P(I)$, and $P(R)$ for each node over time.



$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$



From SIR to Network SIR

A contact network is represented as a graph:

- V : set of nodes / individuals
- E : set of edges / possible contacts
- n : number of nodes in the network

The network structure is encoded by the adjacency matrix $A \in \mathbb{R}^{n \times n}$:

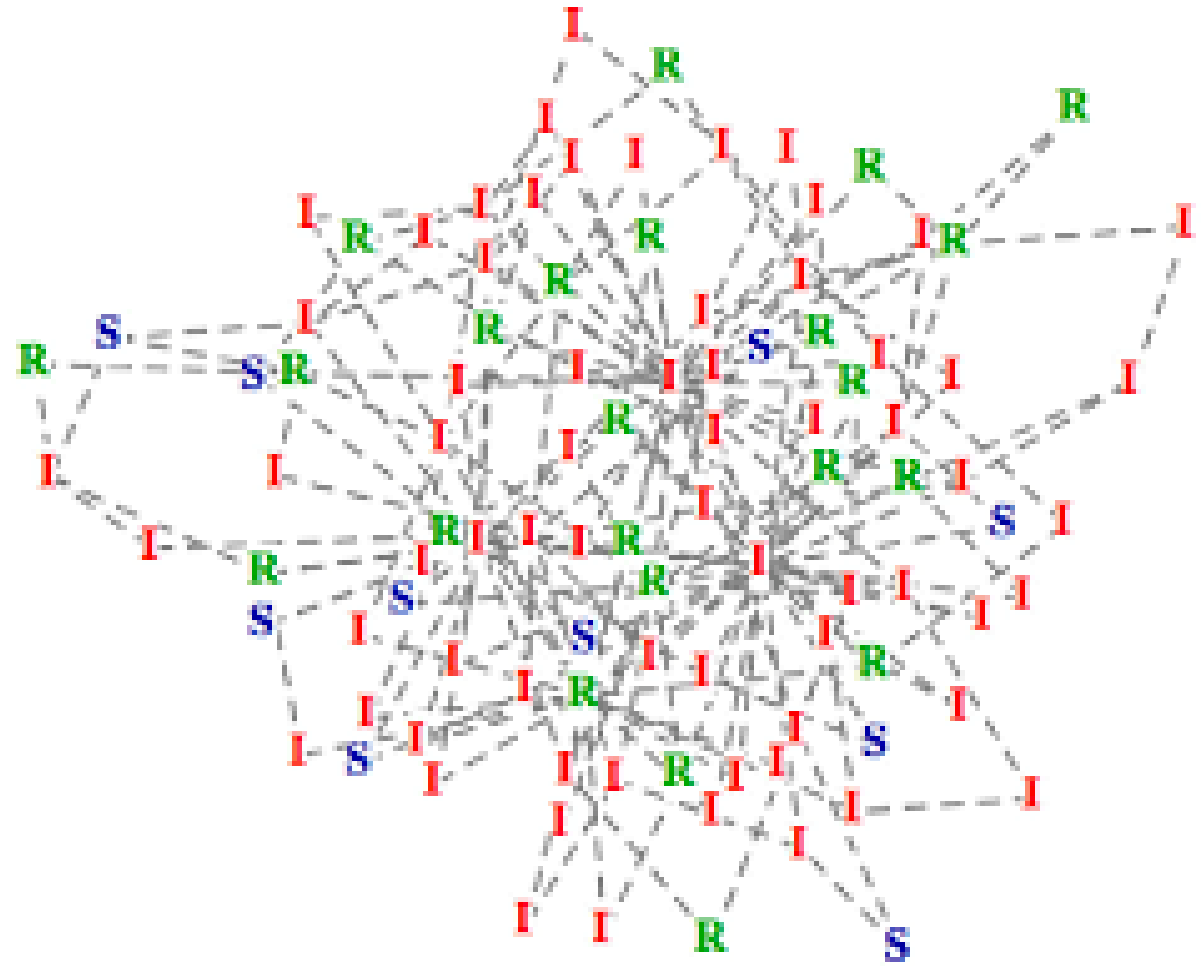
$$A_{ij} = \begin{cases} 1, & \text{if node } i \text{ is connected to node } j \\ 0, & \text{otherwise} \end{cases}$$

Each node has three state probabilities at time t :

$$s_i(t), i_i(t), r_i(t)$$

With $s_i(t) + i_i(t) + r_i(t) = 1$

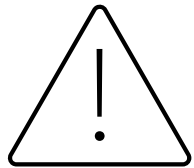
For all nodes together: $s(t), i(t), r(t) \in \mathbb{R}^n$



SIR gives the state transitions; A gives the contact structure.

Why Exact Network SIR Becomes Intractable

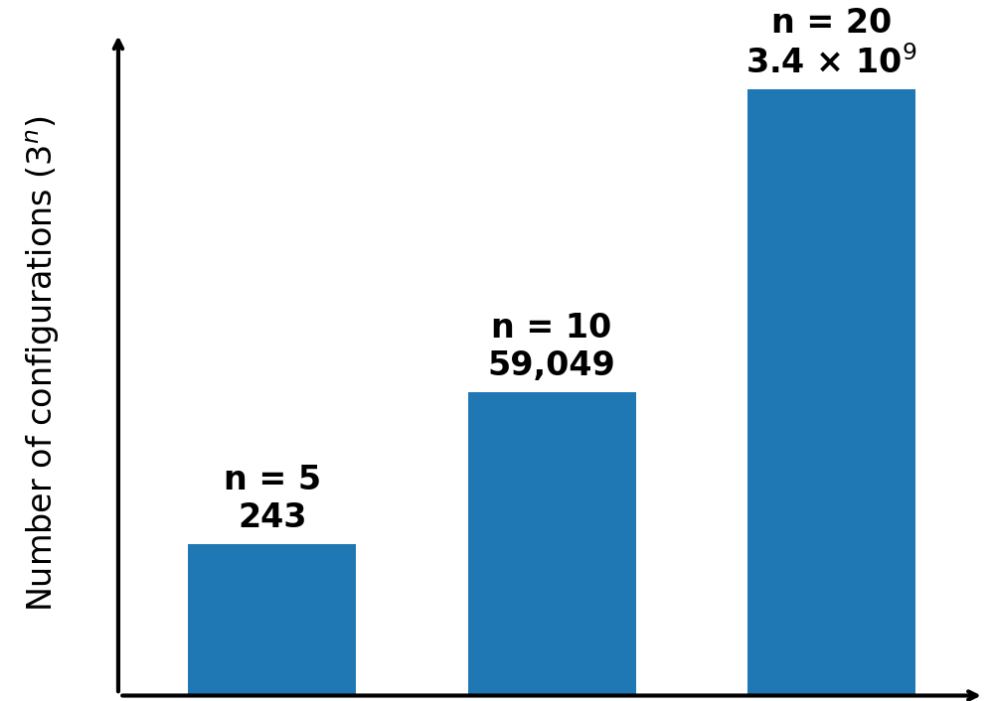
In a network with n nodes, each node can be in one of 3 states:
 S, I, R



The global epidemic state is described by all possible joint configurations of the network:
 3^n possible system states

Exact SIR dynamics require tracking the probability of each global configuration over time.

→ A system of 3^n coupled differential equations, which grows exponentially with network size.



Existing Approximations and Their Tradeoffs

Differential-Equation Approximation

Solves a simplified ODE system instead of the full 3^n -state epidemic process.

Fast but **inaccurate** due to simplifying assumptions

Dynamic Message Passing

Tracks how infection probability flows along edges. Each edge carries a time-dependent message estimating whether infection has passed through it.

Accurate on tree-like networks but **costly** on large dense graphs

Monte Carlo Simulation

Simulates the stochastic epidemic many times and averages the outcomes.

Accurate but **inefficient**, typically requires 10,000+ simulations



The GN-ODE Model

Approximate ODE core · Neural architecture · Datasets and baselines

Why the Approximate SIR ODE Is Not Enough

The approximate ODE gives a structured but simplified view of the epidemic:

$$\frac{dS}{dt} = -\beta(AI_h) \odot S_h$$

$$\frac{dI}{dt} = \beta(AI_h) \odot S_h - \gamma I_h$$

$$\frac{dR}{dt} = \gamma I_h$$

Here, A is the adjacency matrix; $\odot$ is element-wise - captures network structure; AI_h means **infected-neighbor influence**; h denote hidden representations.

To make the SIR dynamics solvable, the model assumes **node-state independence**.

Exact infection term:

$$P(X_j = I \cap X_i = S)$$

Approximate infection term:

$$P(X_j = I)P(X_i = S)$$

- The fixed ODE only uses individual node probabilities.
- It does not explicitly model dependencies between connected node states.
- It has no trainable parameters, so it cannot adapt when the independence approximation is inaccurate.

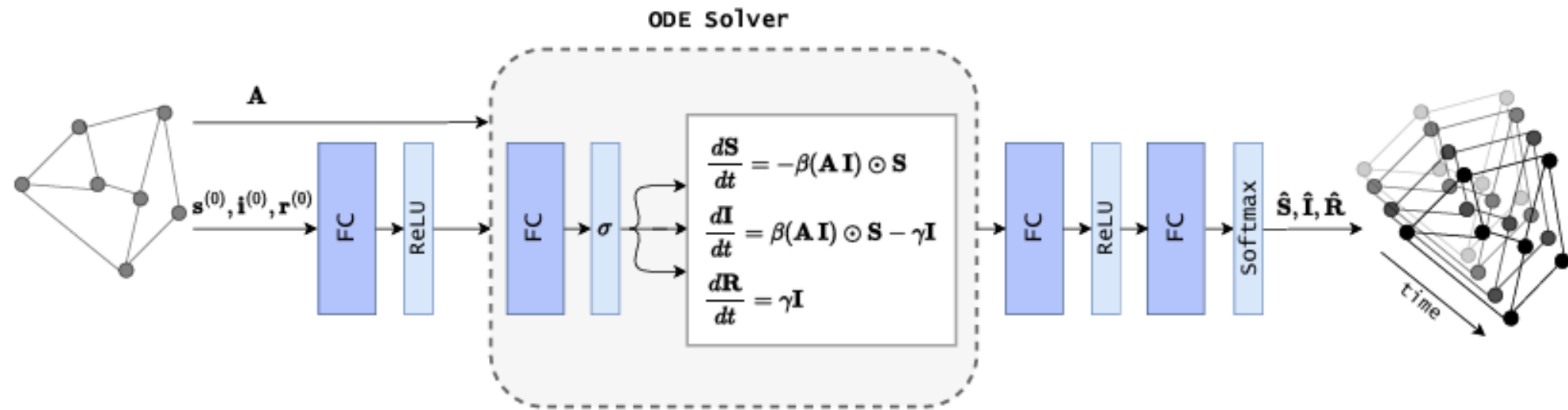


The ODE gives **structure**; the neural network learns **what the independence assumption leaves out**.

GN-ODE Architecture: What Is Learned?



Idea: keep the SIR structure but learn how to refine its dynamics on a graph.



1 Input: Graph A , initial states $s^{(0)}, i^{(0)}, r^{(0)}$, and parameters β, γ

2 Encode: FC + ReLU maps S/I/R states into hidden node representations

3 Evolve: ODE solver updates hidden states using SIR-style graph dynamics

4 Decode: MLP + softmax converts hidden states into node probabilities $\hat{S}^i(t), \hat{I}^i(t), \hat{R}^i(t)$



The final output is the predicted probability that each node is susceptible, infected, or recovered at each time.

Experimental Setup

Datasets

Dataset	karate	dolphins	fb-food	fb-social	openflights	Wiki-Vote	Enron	Epinions
#nodes	34	62	620	1,893	2,905	7,066	33,696	75,877
#edges	78	159	2,102	13,835	15,645	100,736	180,811	405,739
Transitivity	0.256	0.309	0.223	0.057	0.255	0.125	0.085	0.066
Density	0.1390	0.0841	0.0110	0.0077	0.0037	0.0040	0.0003	0.0001
Max. degree	17	12	134	255	242	1065	1383	3044

Experimental protocol

Instances:	200 SIR instances / network
Rates:	$\beta, \gamma \in [0.1, 0.5]$, uniformly sampled
Initial:	2 infected nodes
Ground truth:	10,000 Monte Carlo runs
Split / metric:	60 / 20 / 20; MAE

Baselines

1

DMP

Dynamic Message
Passing

Non-ML probabilistic
approximation; passes
infection-risk messages
along edges.

2

GCN

Graph Convolutional
Network

GNN baseline that
averages information
from neighboring nodes.



GN-ODE

Graph Neural ODE

Proposed model; combines
SIR-inspired ODE dynamics
with learned graph
representations.

Experimental Setup

Datasets

Dataset	karate	dolphins	fb-food	fb-social	openflights	Wiki-Vote	Enron	Epinions
#nodes	34	62	620	1,893	2,905	7,066	33,696	75,877
#edges	78	159	2,102	13,835	15,645	100,736	180,811	405,739
Transitivity	0.256	0.309	0.223	0.057	0.255	0.125	0.085	0.066
Density	0.1390	0.0841	0.0110	0.0077	0.0037	0.0040	0.0003	0.0001
Max. degree	17	12	134	255	242	1065	1383	3044

Experimental protocol

Instances:	200 SIR instances / network
Rates:	$\beta, \gamma \in [0.1, 0.5]$, uniformly sampled
Initial:	2 infected nodes
Ground truth:	10,000 Monte Carlo runs
Split / metric:	60 / 20 / 20; MAE

Baselines

1

DMP

Dynamic Message
Passing

Non-ML probabilistic
approximation; passes
infection-risk messages
along edges.

2

GCN

Graph Convolutional
Network

GNN baseline that
averages information
from neighboring nodes.



GN-ODE

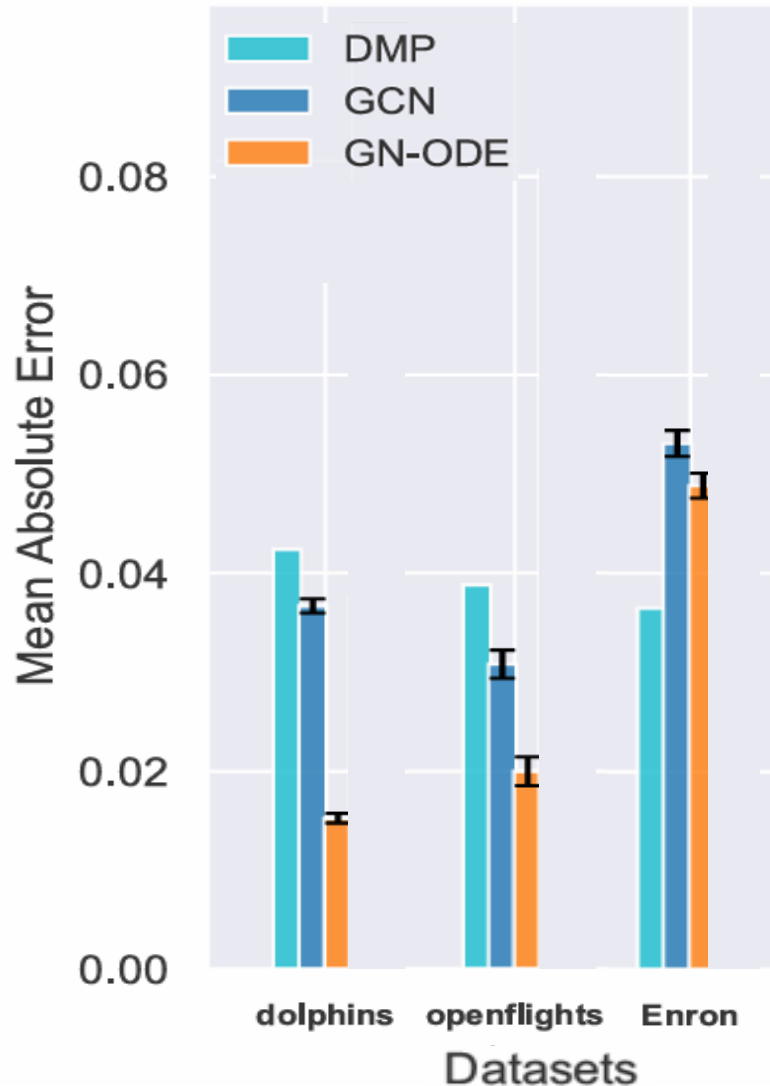
Graph Neural ODE

Proposed model; combines
SIR-inspired ODE dynamics
with learned graph
representations.

Results and Conclusions

Prediction accuracy · Scalability · Generalization to unseen settings · Final takeaway

Does GN-ODE Predict Epidemic Spreading **More Accurately?**

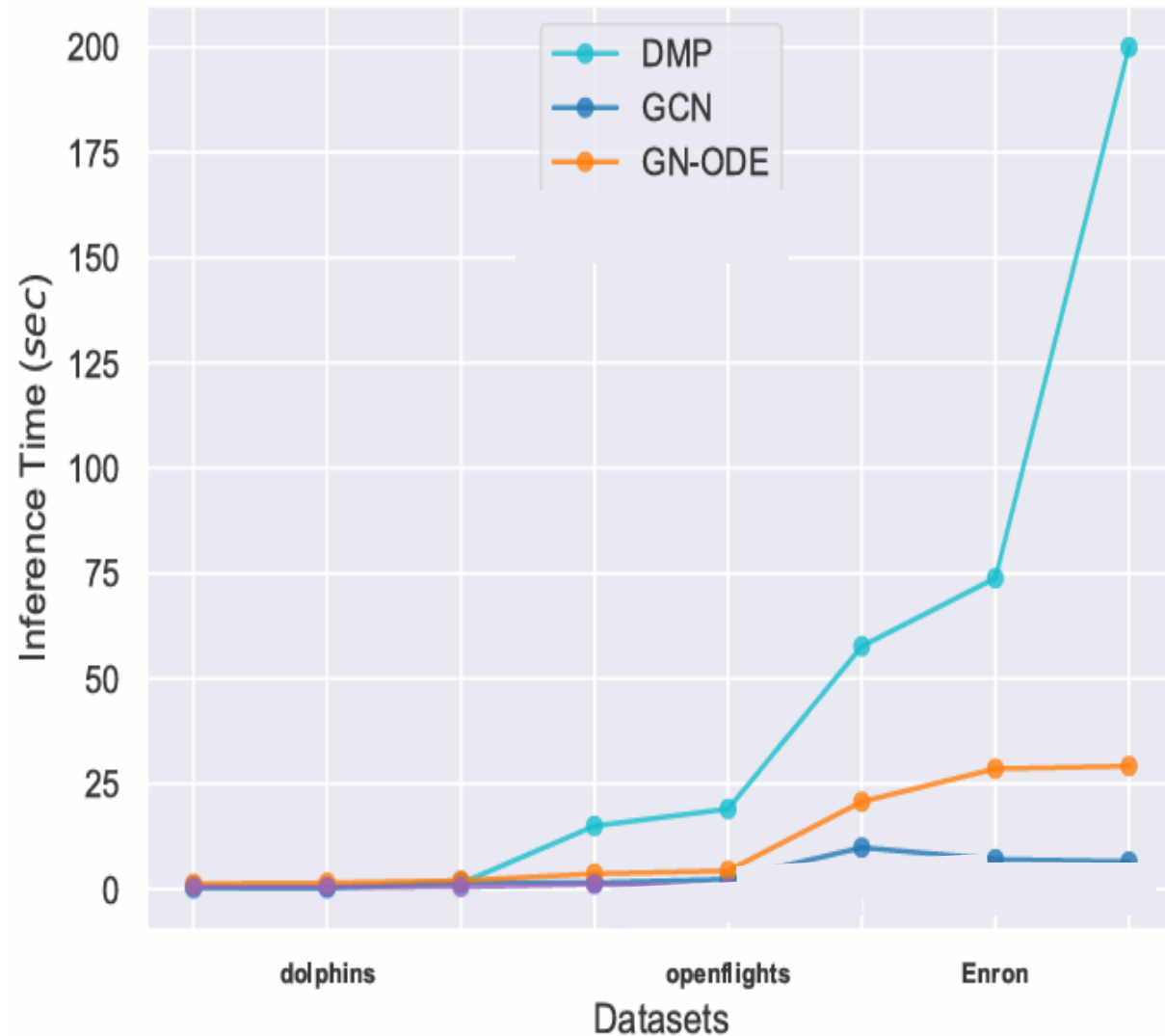
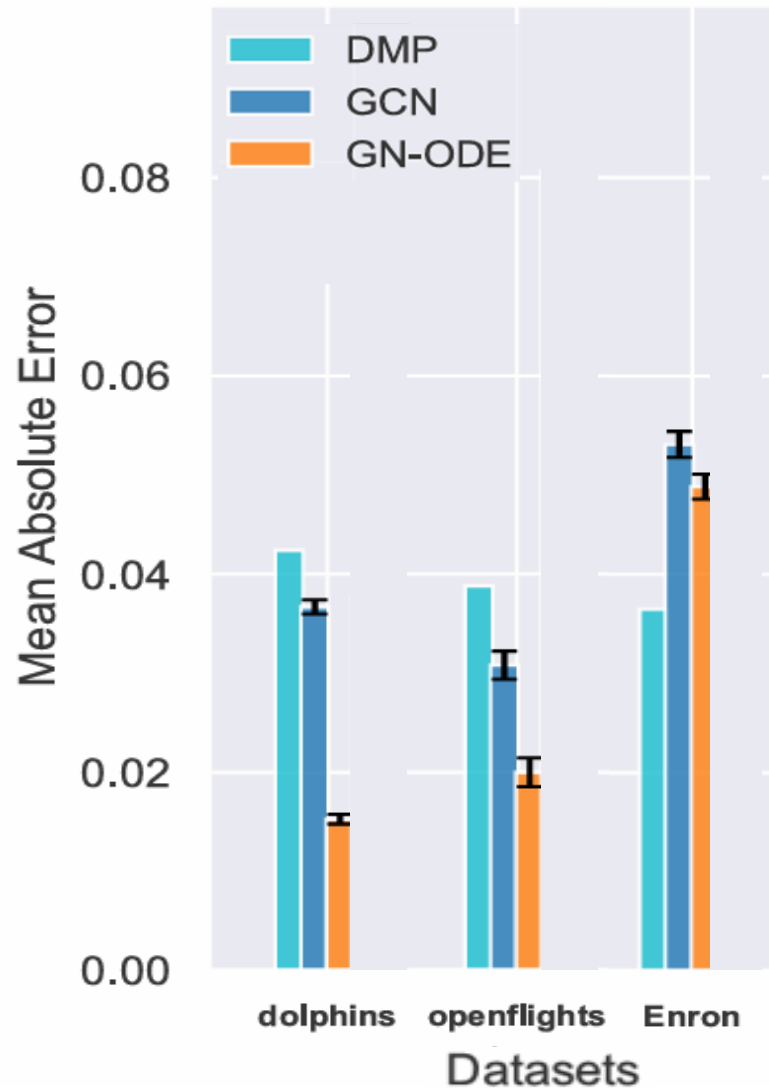


- **Setting:** Same-network SIR prediction; train/test simulations use different infection and recovery parameters.
- **Result:** GN-ODE has the lowest error on Dolphins and OpenFlights, while DMP is best on Enron.
- **Interpretation:** Learned ODE dynamics help on several networks, but classical DMP remains competitive, especially on larger graph structures.

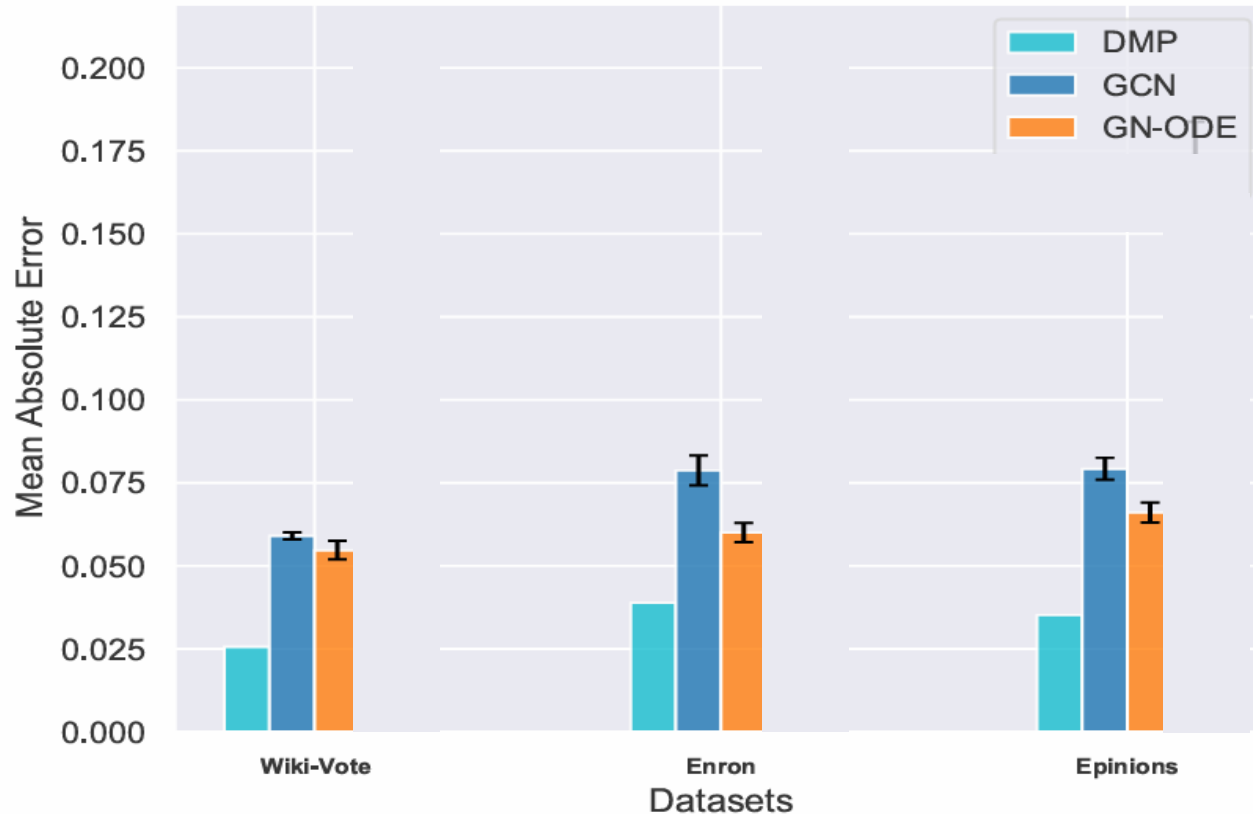


GN-ODE improves prediction on 2 of the 3 selected datasets, but the result is not universal: network structure matters.

Does GN-ODE Predict Epidemic Spreading More Accurately?



Can GN-ODE Generalize to Larger Unseen Networks?



GN-ODE generalizes better than the neural baseline GCN on unseen large networks, but DMP remains the accuracy benchmark. The value of GN-ODE is the [trade-off](#): learned epidemic dynamics with better scalability than purely combinatorial inference.

- **Setting:** Trained on smaller networks, then tested on larger unseen networks: Wiki-Vote, Enron, and Epinions.
- **Scale jump:** The test networks are much larger: Wiki-Vote has 7k nodes, Enron 34k nodes, and Epinions 76k nodes.
- **Result:** Among the neural models shown, GN-ODE consistently has lower MAE than GCN on all three unseen networks.
- **Important nuance:** DMP still achieves the lowest error, but it is a combinatorial method and becomes much slower on large graphs.

Conclusion

Goal: assess whether Neural ODEs can model continuous epidemic dynamics on networks more effectively than fixed approximations or standard GNNs.

1

ODE perspective: an ODE describes how a state changes over time. Starting from an initial value, following this rule gives the trajectory.

2

Neural ODE perspective: Neural ODE learns the vector field f_θ , meaning the local rule of motion. The ODE solver integrates this learned rule into a full hidden-state trajectory.

3

GN-ODE perspective: GN-ODE applies this idea to epidemic spreading on graphs. It keeps SIR-inspired dynamics but makes them learnable through neural graph transformations.

4

Empirical takeaway: GN-ODE often improves over standard neural baselines such as GCN, but DMP can still be more accurate on some large graphs.



GN-ODE shows how scientific structure and neural learning can work together to model complex network dynamics more effectively.

References

Chen et al. (2018). *Neural Ordinary Differential Equations*. NeurIPS.

Kosma et al. (2023). *Neural Ordinary Differential Equations for Modeling Epidemic Spreading*. TMLR.

Kim et al. (2017). *Coupling effects on turning points of infectious diseases epidemics in scale-free networks*. BMC Bioinformatics.

De Witte (2019). *Who you don't know: Stanford economist examines how a weak social network can explain inequality, social immobility*. Stanford Report.

Ray, N. (2025). *Ordinary Differential Equations for Data Science* [Lecture notes]. Katholische Universität Eichstätt–Ingolstadt.

Use of AI Tools

I used AI assistance (ChatGPT) during the preparation of this presentation for code generation related to visualizations, language polishing, and improving the clarity and flow of the slides. All scientific ideas, content decisions, interpretation of results, and conclusions were developed and verified by me.

Thank You